

ICML 2024 | Transformer 真正的优势是并行，而不是规模

哥伦比亚大学与纽约大学:对数深度的 Transformer 等价于大规模并行计算(MPC)

Transformer 几乎主导了所有序列建模任务,但一个最基础的问题始终没有令人满意的答案:它究竟强在哪里?过去的理论大致分两条线。一条证明了 Transformer 的“通用性”,却只对规模大得不切实际的模型成立,无法说明哪些任务能被高效地解决;另一条固定深度、增大上下文长度,在这个设定下,连括号匹配、布尔公式求值这类基础算法任务都被证明做不到。两幅图景都没能精确指出:到底是什么属性,把 Transformer 和 RNN 这样的其他架构区分开来。

这篇由 Clayton Sanford、Daniel Hsu 与 Matus Telgarsky 完成、发表于 ICML 2024 的工作给出的答案是:并行。自注意力本质上是一次并行操作,任意两个 token 都能在一层之内、通过 query 与 key 的内积直接交互;而 RNN 只能把信息沿着序列一步一步地传递。作者把这种并行性对应到理论计算机科学中的大规模并行计算模型(Massively Parallel Computation,简称 MPC),也就是刻画 MapReduce 这类分布式系统的抽象:许多台机器各自持有少量数据,在同步的一轮轮通信里做本地计算、互相收发消息。

沿着这个视角,论文证明了一个双向的等价:对数深度的 Transformer 与常数轮数的 MPC 协议,在算法能力上完全对应。这个等价既解释了 Transformer 的强,也划出了它的边界。作者还设计了一个干净的合成任务——k-hop induction heads,在理论上证明 Transformer 只需对数深度即可求解,而 RNN、状态空间模型(如 Mamba)以及次二次注意力(如 Performer)都做不到;随后训练真实模型,观察到它们恰好落在理论预测的深度阈值上。

论文 :<https://arxiv.org/abs/2402.09268> 代码 :<https://github.com/chsanford/hop-induction-heads>

把一层自注意力看成一轮并行通信

MPC 是分布式算法研究里的标准模型:数据被切分到许多台内存受限的机器上,计算按同步的“轮”推进,每一轮里每台机器先做本地计算,再把带地址的消息发给其他机器。它的能力用轮数来衡量,轮数越少,算法越“并行”。论文的核心主张是:一层自注意力恰好能模拟 MPC 的一轮。下面这张图给出了单个注意力头是怎么承担这件事的。

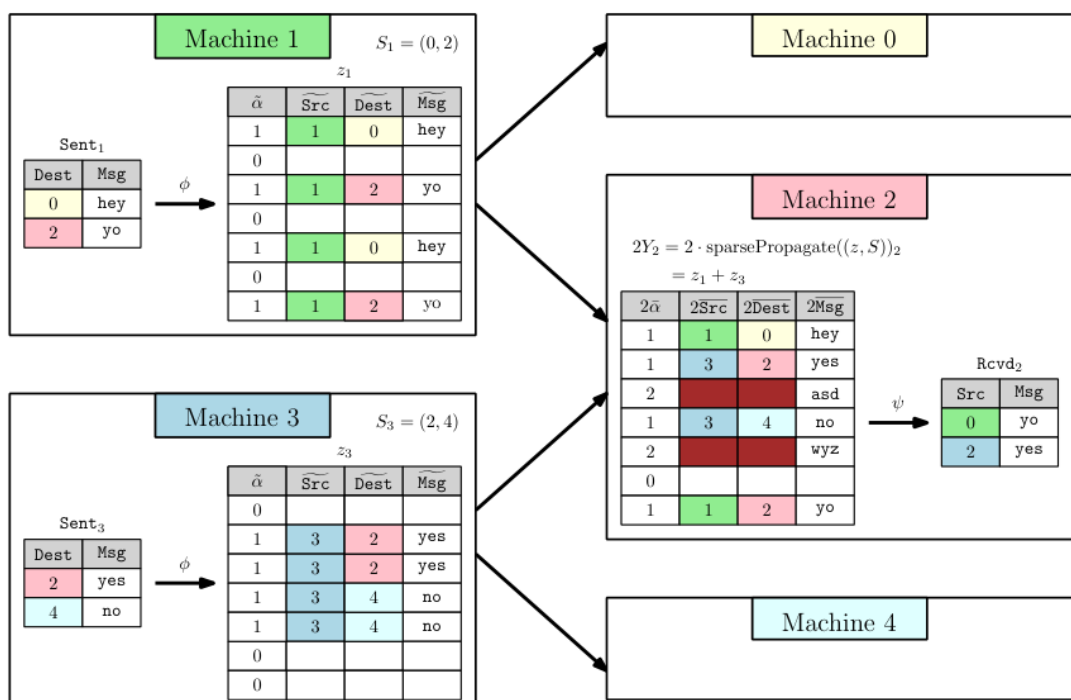


图 2 消息路由示例:机器 1 与机器 3 向机器 2 发送消息。每条消息通过“多重哈希”被编码进嵌入空间中多个固定位置的“数据包”里;这种冗余保证每条消息至少有一次单独出现,机器 2 因此能从汇聚到的 Y_2 中把 Rcvd₂ 解码出来。

图里的编码(ϕ)把每台机器要发送的内容映射成嵌入向量,注意力完成汇聚后,解码(ψ)再从机器 2 收到的行里读回它该接收的消息。之所以要把同一条消息放进多个数据包,是因为只有“至少单独出现一次”才能保证解码无歧义——这正是构造能在小嵌入维度下运作的原因,也是把注意力和一轮通信精确对齐的关键一步。

k-hop induction heads:一个能分辨架构的任务

为了把结论落到实处,作者设计了 k-hop induction heads 任务。标准的 induction heads 要求模型完成一个 bigram:预测当前 token 上一次出现时,紧跟其后的那个 token。k-hop 版本把这一步串起来:用一次补全决定下一次要补全哪个 bigram,如此重复 k 次。评测用长度 $N = 100$ 的序列、大小为 4 的字母表($|\Sigma| = 4$),并让 k 在 $\{0, \dots, 16\}$ 之间变化,同一个模型每次随机处理一个 k。直觉上这像是需要 k 个串行步骤,但并行架构能把它折叠进对数多的层里——论文给出的构造深度恰好是 $L = \lfloor \log_2 k \rfloor + 2$,而且宽度为常数。

实验:训练出的模型恰好卡在对数阈值上

理论预测能不能在真实训练里看到?作者训练了深度 2 到 6 的 Transformer,测量它们在多任务设定下随 k 增大的逐 token 分类误差。结果异常干净,下图把它完整地画了出来。

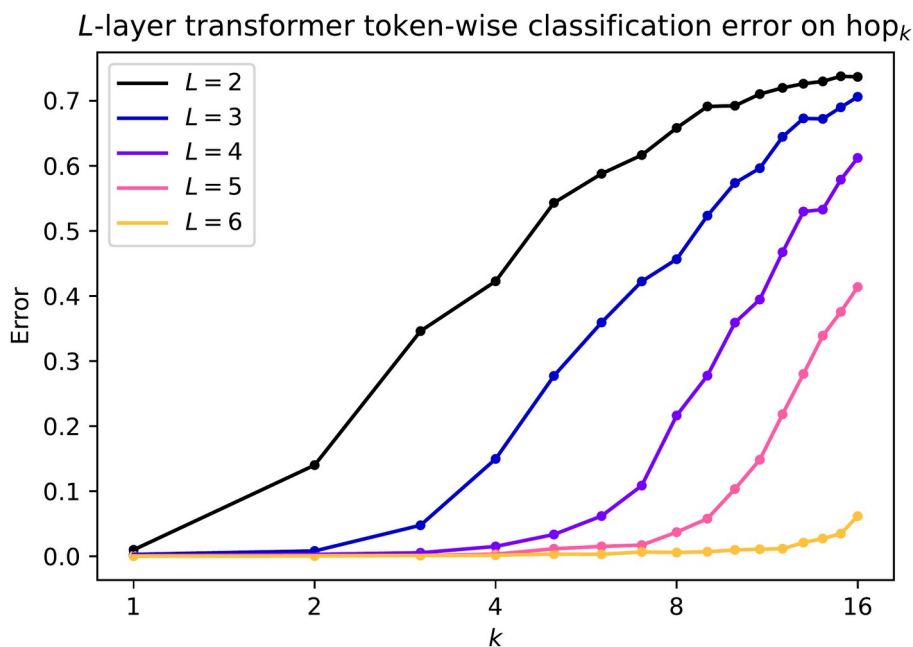


图3 深度 $L \in \{2, 3, 4, 5, 6\}$ 的 Transformer 在 hop_k 上的逐 token 分类误差(k 从 1 到 16)。每增加一层,能被小误差学会的最大 k 大约翻一倍:曲线整体向右平移一档,经验阈值恰好落在 $\lfloor \log_2 k \rfloor + 2$ 。

读法很直接:每增加一层,能被学会的最大 k 大约翻倍。6 层的模型在所有 $k \leq 16$ 上都表现良好,5 层能覆盖到 $k \leq 8$,4 层覆盖到 $k \leq 4$ ——可解的 k 随深度指数增长,和构造性理论完全吻合,经验边界正是 $\lfloor \log_2 k \rfloor + 2$ 。几个消融进一步支撑了这一点:把 (m, H) 从 $(128, 4)$ 增大到 $(256, 8)$,深度- k 边界几乎不动,说明依赖的是深度而非单纯的规模;在只有 1000 到 3000 个样本的有限数据设定下,更深的模型泛化得更好,暗示了一种适合组合式任务的归纳偏置;而拆开训练好的网络会发现,注意力矩阵正在计算证明里用到的中间 find_j 指针,学到的机制和手工构造惊人地一致。

表 1 求解 $k\text{-hop induction heads}$ 所需的深度:Transformer 是对数的,串行架构是线性的。

架构	求解 $k\text{-hop}$ 所需深度
Transformer(自注意力)	$\lfloor \log_2 k \rfloor + 2$ (对数)
多层 RNN / 状态空间模型(Mamba)	$\geq k$ (线性)
次二次注意力(如 Performer)	$\geq k$ (线性)

结论与边界

这篇论文想让人记住的一点是:Transformer 在一种精确的意义上是并行计算机。作者把这个直觉钉死为一条定理——对数深度的 Transformer 等价于常数轮数的大规模并行计算,而这条等价既解释了它的能力,也预言了一条清晰的分界:在 $k\text{-hop induction heads}$ 上,Transformer 只需对数于 k 的深度就能求解,RNN、Mamba 这样的状态空间模型以及 Performer 这样的高效注意力近似,却都需要线性于 k 的深度。当然,结论也有边界:任务是合成的,前向构造里的常数并不小,而反方向“近似最优”的那部分依赖于一个尚未证明的 MPC 下界猜想。即便如此,它给出的信息很明确——Transformer 真正的优势,不只是规模,而是同时做很多件事的能力。